

*System
Software Vs.
Application
Software*

Computer software

Computer software, or just software, is a general term primarily used for set of instructions such as computer programs and other kinds of information read and written by computers.

Types of Software

- System Software
- Application Software

A computer system consists of hardware and software. The computer hardware cannot perform any task on its own. It needs to be instructed about the tasks to be performed. Software is a set of programs that instructs the computer about the tasks to be performed.

TYPES OF SOFTWARE

Software can be broadly classified in two categories:

1. **System Software**(System software provides the basic functions that are performed by the computer. It is necessary for the functioning of a computer.)
2. **Application Software**(Application software is used by the users to perform specific tasks. The user may choose the appropriate application software, for performing a specific task, which provides the desired functionality.)

What is System Software ?



- A software that provides platform to other softwares.
- Also called as by default or in built software.
- These are commonly prepared by the computer manufacturers.
- These softwares consist of programs written in low-level languages
- Used to interact with the hardware at a very basic level.
- System software serves as the interface between the hardware and the end users.



Software hierarchy

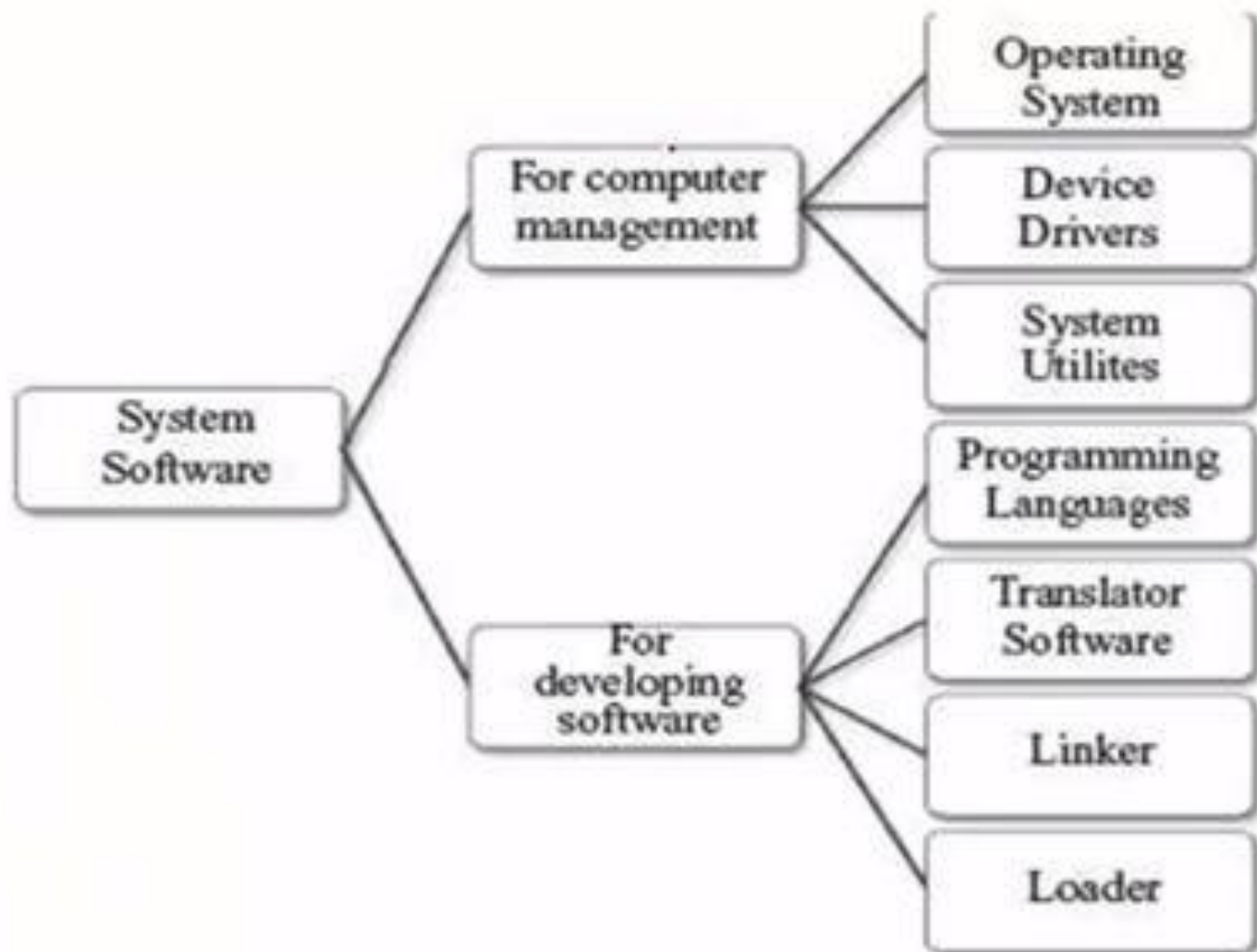
SOFTWARES OF COMPUTER

○ SYSTEM SOFTWARE

- System software provides basic functionality to the computer. System software is required for the working of computer itself. The user of computer does not need to be aware about the functioning of system software, while using the computer. For example, when you buy a computer, the system software would also include different device drivers.
- *The purposes Of system software are:*
 - To provide basic functionality to computer,
 - To control computer hardware
 - To act as an interface between *user, application software and computer hardware.*

Important Features of System Software

- Closeness to the system ✓
- Fast speed
- Difficult to manipulate
- Written in low level language
- Difficult to design



System software

SOFTWARES OF COMPUTER

- **Operating System**

- Operating System (OS) is an important part of a computer. OS intermediates between the user of a computer and the computer hardware.

- **Device Driver**

- A device driver acts as a translator between the hardware and the software that uses the devices. In other words, it intermediates between the device and the software, in order to use the device.
- Some devices that are commonly connected to the computer are—keyboard, mouse, hard disk, printer, speakers, microphone, joystick, webcam, scanner, digital camera, and monitor.

SOFTWARES OF COMPUTER

- **System Utilities** ✓
- System utility software is required for the maintenance of computer. System utilities are used for supporting and enhancing the programs and the data in computer. Some system utilities may come embedded with OS and others may be added later on.
- **Some examples of system utilities are:**
 - *Anti-virus utility to scan computer for viruses*
 - *Data Compression utility to compress the files*
 - *Cryptographic utility to encrypt and decrypt files.*
 - *Disk Compression utility to compress contents of a disk for increasing the capacity of a disk.*

SOFTWARES OF COMPUTER

- **Disk Partitioning** to divide a single drive into multiple logical drives. Each drive is then treated as an individual drive.
- **Disk Cleaners** to find files that have not been used for a long time. It helps the user to decide what to delete when the hard disk is full.
- **Backup Utility** to make a copy of all information stored on the disk. It also restores the backed up contents in case of disk failure.
- **System Profiling Utility** provides detailed information about the software installed on the computer and the hardware attached to it.
- **Network Managers** to check the computer network and to log events.

SOFTWARES OF COMPUTER

- **Programming Languages** ✓
- A Programming Language consists of a set of vocabulary and grammatical rules, to express the computations and tasks that the computer has to perform. Programming Language usually refers to high-level languages like COBOL, BASIC, FORTRAN, C, C++, Java etc.
- **Programming languages fall into three categories.**
- *Machine Language* is what the computer can understand but it is difficult for the programmer to understand. Machine languages consist of numbers only. Each kind of CPU has its own unique machine language.
- *Assembly Language* falls in between machine language and high-level language. They are similar to machine language, but easier to program in, because they allow the programmer to substitute names for numbers.
- **High-level Language** is easier to understand and use for the programmer but difficult for the computer.

SOFTWARES OF COMPUTER

○ Translator Software

○/

- *Translator software* is used to convert a program written in **high-level language** and **assembly language** to a form that the computer can understand. Translator software converts a program written in assembly language, and high-level language to a machine-level language program.
- The translated program is called the *object code*.
- *There are three different kind of translator software:*
 - Assembler
 - Compiler
 - Interpreter

SOFTWARES OF COMPUTER

- **Linker**

- *Linker is a program that links several object ~~modules and libraries~~ to a single executable program;*

- **Loader**

- *The loader software is used to load and re-locate the executable program in the main memory.*
- Software has to be loaded into the main memory during execution. Loader assigns storage space to the program in the main memory for execution.

SOFTWARES OF COMPUTER

○ APPLICATION SOFTWARE

- The software that a user uses for accomplishing a specific task is the *application software*.
- Application software may be a single program or a set of programs. A set of programs that are written for a specific purpose and provide the required functionality is called software package.
- Application software is written for different kinds of applications—graphics, word processors, media players, database applications, Telecommunication, accounting purposes etc.

What is Application Software ?

- A software created for a specific purpose.
- It is generally a program or collection of programs used by end users.
- It can be called an application or simply an app.
- A software which is developed to help the user to perform specific tasks is called application software.

SOFTWARES OF COMPUTER

- *Suite of Software* having Word Processor, Spreadsheet and Presentation Software: Some examples are MS-Office, Google Docs, Sun Openoffice, Apple iWork.
- *CAD/CAM Software*: To assist in architectural design. (e.g. AutoCAD, Autodesk)
- *Geographic Information Systems*: It captures, stores, analyzes, manages, and presents data, images and maps that are linked to different locations. (e.g. ArcGIS)
- *Web Browser Software*: To access the World Wide Web to search documents, sounds, images etc. (e.g. Internet Explorer, Netscape Communicator, Chrome).

System Software

System Software includes the Operating System and all the utilities that enable the computer to function.

System software is a term referring to any computer software which manages and controls the hardware so that application software can perform a task.

Example:

Operating Systems, Compiler, Loader, Linker, Interpreter.

Application Software

Application Software includes programs that do real work for user.

Example:

Payroll systems, Inventory Control, Manage student database, Word Processor, Spreadsheet and Database Management System etc.,

System software Vs. Application Software

System software is a Software designed to operate the computer hardware and to provide and maintain a platform for running application software.

System Software:

1. Operating System

- Operating System is a software, which makes a computer to actually work.
- It is the software the enables all the programs we use.
- The OS organizes and controls the hardware.
- OS acts as an interface between the application programs and the machine hardware.
- Examples: Windows, Linux, Unix ,DOS

System Software

(contd...):

2. Compiler: A compiler is a program that reads a program in one language – the source language and translates into an equivalent program in another language – the target language (Machine understandable Language).

Source Language

'C' language

'Pascal' language

FORTRAN language

C++ language

ADA language

Target Languages

Machine

Understandable
language

What is Compiler ?



What is Compiler ?

- In a compiler, the source code is translated to object code successfully if it is free of errors.
- The compiler specifies the errors at the end of compilation with line numbers when there are any errors in the source code.
- The errors must be removed before the compiler can successfully recompile the source code again.

System Software

(contd...):

3. **Loader**: A loader is the part of an operating system that is responsible for loading programs into memory, preparing them for execution and then executing them.

The loader is usually a part of the operating system's kernel and usually is loaded at system boot time and stays in memory until the system is rebooted, shut down, or powered off.

System Software

(contd...):

4. Linker: A linker or link editor is a program that takes one or more objects generated by compilers and assembles them into a single executable program.

Linkers can take objects from a collection called a *library*. The objects are program modules containing machine code and information for the linker.

System Software

(contd...):

5. Interpreter: An interpreter is a computer program that translates and executes instructions written in a computer programming language line-by-line, unit by unit etc.,

An interpreter needs to be able to analyze, or parse, instructions written in the source language.

Example: Lisp systems, etc.,

What is Interpreter ?

- The translation of single statement of source program into machine code is done by language processor and executes it immediately before moving on to the next line is called an interpreter.
- If there is an error in the statement, the interpreter terminates its translating process at that statement and displays an error message.
- The interpreter moves on to the next line for execution only after removal of the error.
- An Interpreter directly executes instructions written in a programming or scripting language without previously converting them to an object code or machine code.
- **Example:** Perl, Python and Matlab.

System Software

(contd.):

6. Device Driver is a computer program that allows higher level computer programs to interact with a hardware device.

- Communicates with the device through bus or subsystems to which the hardware is connected.
- When a calling program invokes a routine in the driver, the driver issues commands to the device. Once the device sends data back to the driver, the driver may invoke routines in the original routine.

System Software

(contd.):

6. Device Driver

- available for printers, displays, CD-ROM readers, diskette drives, and so on
- hardware dependent and operating system specific.

System Software

(contd...):

6. Macroprocessor

A macro processor is a program that reads a file (or files) and scans them for certain keywords. When a keyword is found, it is replaced by some text.

The keyword/text combination is called a macro. Similar to
function

Macros

- To relieve programmers of the need to repeat identical parts of their program, operating systems provide macro facility, which permits the programmer to define an abbreviations for a part of his program.

Application

Software:

1. Word Processors:

Word processing is a tool that helps user in creating, editing, and printing documents. Word processors will normally have the following capabilities built into them:

- Spell checking
- Standard layouts for normal documents
- Have some characters appear in bold print, italics, or underlined
- Center lines, make text line up on the left side of the paper, or the right side of the paper
- Save the document so it can be used again
- print the document.

Examples: WordPerfect and Microsoft Word

Application Software (contd...):

2. **Spreadsheets:** The spreadsheet packages are designed to use numbers and formulas to do calculations with ease. Examples of spreadsheets include:

- Budgets
- Payrolls
- Grade Calculations
- Address Lists

The most commonly used spreadsheet programs are Microsoft Excel and Lotus 123.

Application Software (contd...):

3. Graphic Presentations: The presentation programs can make giving presentations and using overheads easier. Other uses include:

- Slide Shows
- Repeating Computer Presentations on a computer monitor
- Using Sound and animation in slide shows

The most recognized graphic presentation programs are Microsoft PowerPoint and Harvard Graphics.

Application

Software

4. Database Management System (DBMS): (contd...):

- A DBMS is a software tool that allows multiple users to store, access, and process data into useful information.
- Database programs are designed for these types of applications:
 - Membership lists
 - Student lists
 - Grade reports
 - Instructor schedules

All of these have to be maintained so you can find what you need quickly and accurately.

- Example: Microsoft Access, dBASE, Oracle.

System software Vs. Application Software

- System software gets installed when the operating system is installed on the computer
 - System software includes programs such as compilers, debuggers, drivers,
- application software is installed according to the requirements of the user
- application software includes media players, word processors, and spreadsheet programs
- users interact with application software while doing different activities.
- there may be a number of

Compiler Vs. Interpreter

- Scans the entire program and translates it as a whole into machine code.
- It takes large amount of time to analyze the source code but the overall execution time is comparatively faster.
- Generates intermediate object code which further requires linking, hence requires more memory.
- It generates the error message only after scanning the whole program. Hence debugging is comparatively hard.
- Programming language like C, C++ use compilers.

Translates program one statement at a time.

It takes less amount of time to analyze the source code but the overall execution time is slower.

No intermediate object code is generated, hence are memory efficient.

Continues translating the program until the first error is met, in which case it stops. Hence debugging is easy.

Programming language like Python, Ruby use interpreters.

Examples of Machine Level Language

- High-level languages, such as Swift and C++ must be compiled into machine language before the code is run on a computer.
- The ASCII value for the letter "A" is 01000001 in machine code, but this data is displayed as "A" on the screen.
- An image may have thousands or even millions of binary values that determine the color of each pixel.

Why Language Processors are needed ?

- Assembly language is machine dependent yet mnemonics that are being used to represent instructions in it are not directly understandable by machine and high Level language is machine independent.
- A computer understands instructions in machine code, i.e. in the form of 0s and 1s.
- It is a tedious task to write a computer program directly in machine code.
- The programs are written mostly in high level languages like Java, C++, Python etc. and are called **source code**.
- These source code cannot be executed directly by the computer and must be converted into machine language to be executed.
- Hence, a special translator system software is used to translate the program written in high-level language into machine code is called **Language Processor** and the program after translated into 

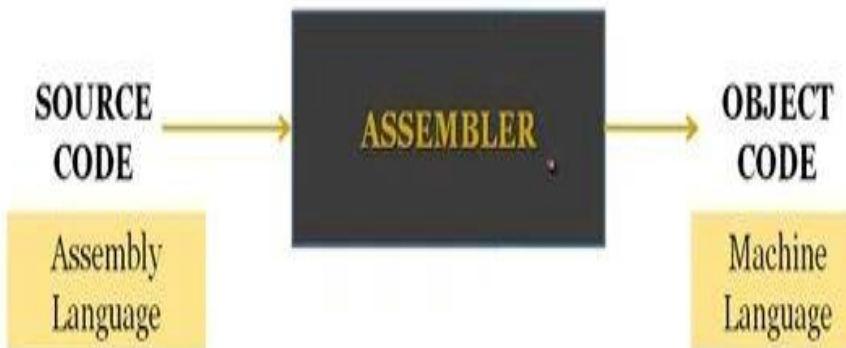
What is Assembly Level Language ?

- A low-level programming language for microprocessors and other programmable devices.
- It is not just a single language, but rather a group of languages.
- An assembly language implements a symbolic representation of the machine code needed to program a given CPU architecture.
- Assembly language is also known as assembly code.
- The term is often also used synonymously with 2GL.
- It is the most basic programming language available for any processor.
- With assembly language, a programmer works only with operations that are implemented directly on the physical CPU.

What is Assembler ?

- The Assembler is used to translate the program written in Assembly language into machine code.
- The source program is a input of assembler that contains assembly language instructions.
- The output generated by assembler is the object code or machine code understandable by the computer.

What is Assembler ?



- Schematic of program execution

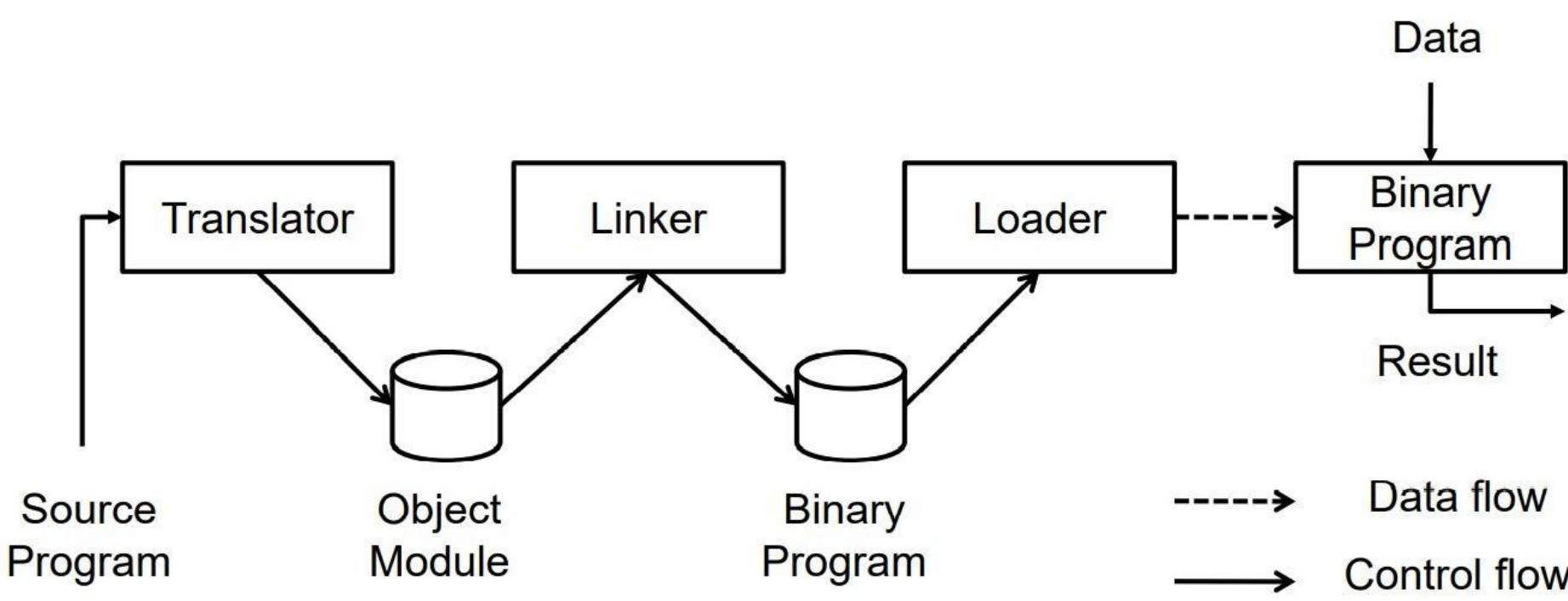
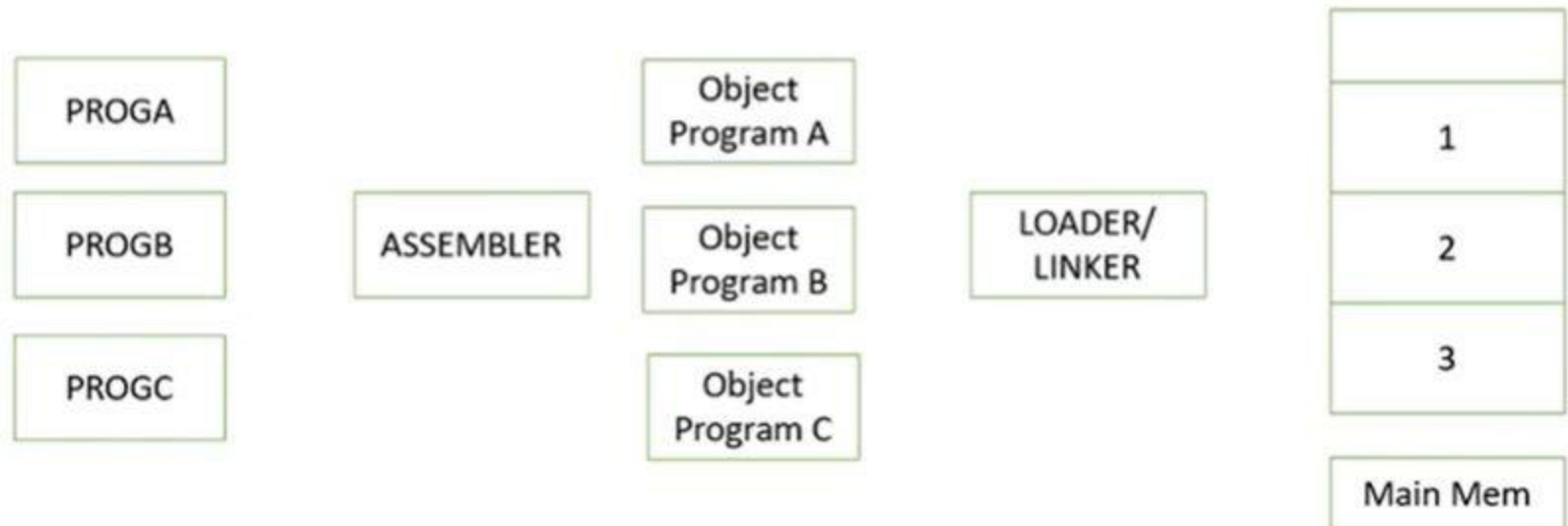
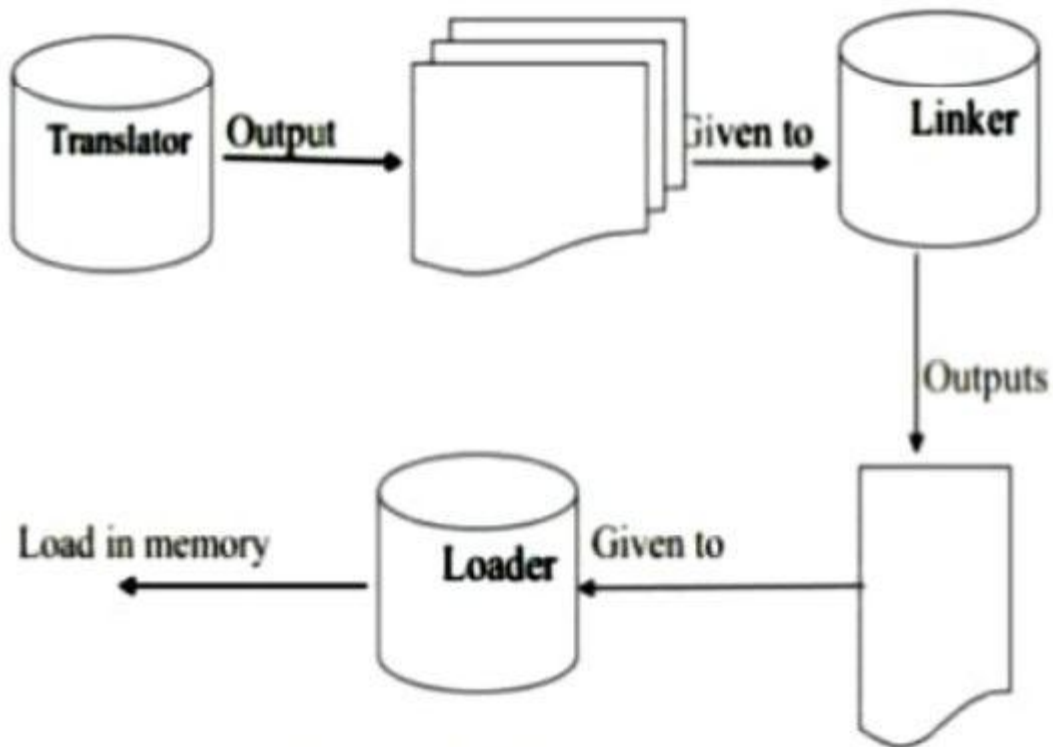


Figure: Schematic of program execution

Program Linking

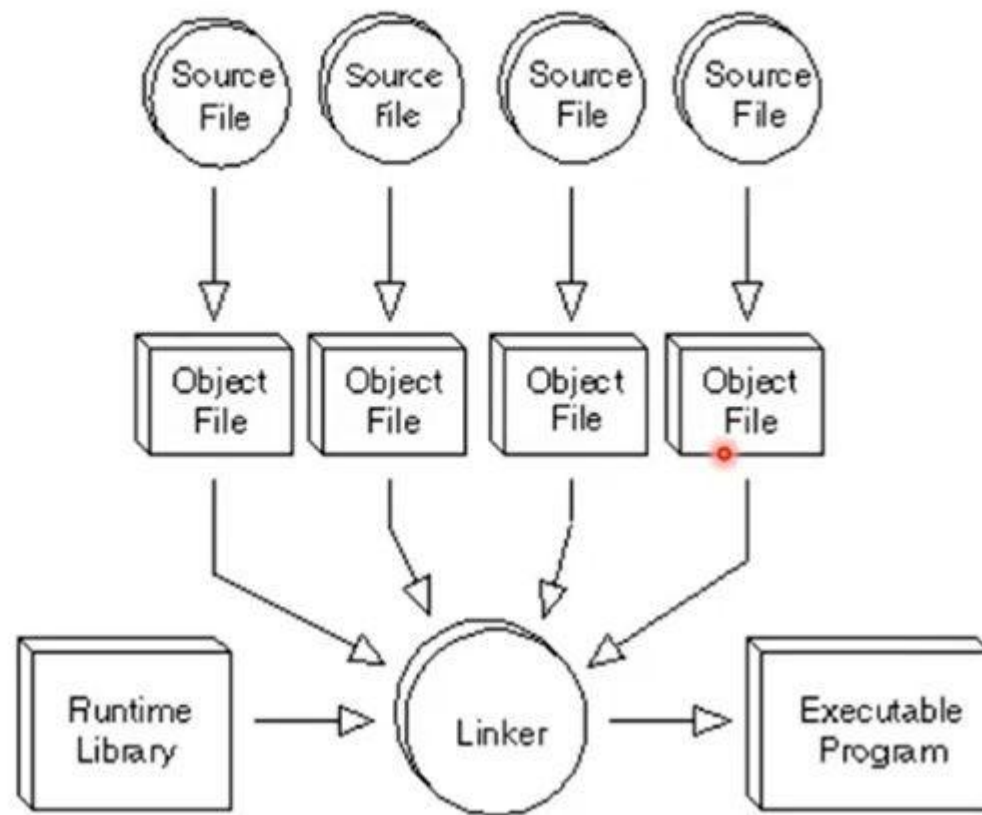
- three controls sections.
 - Assembled **together** or assembled **independently**.



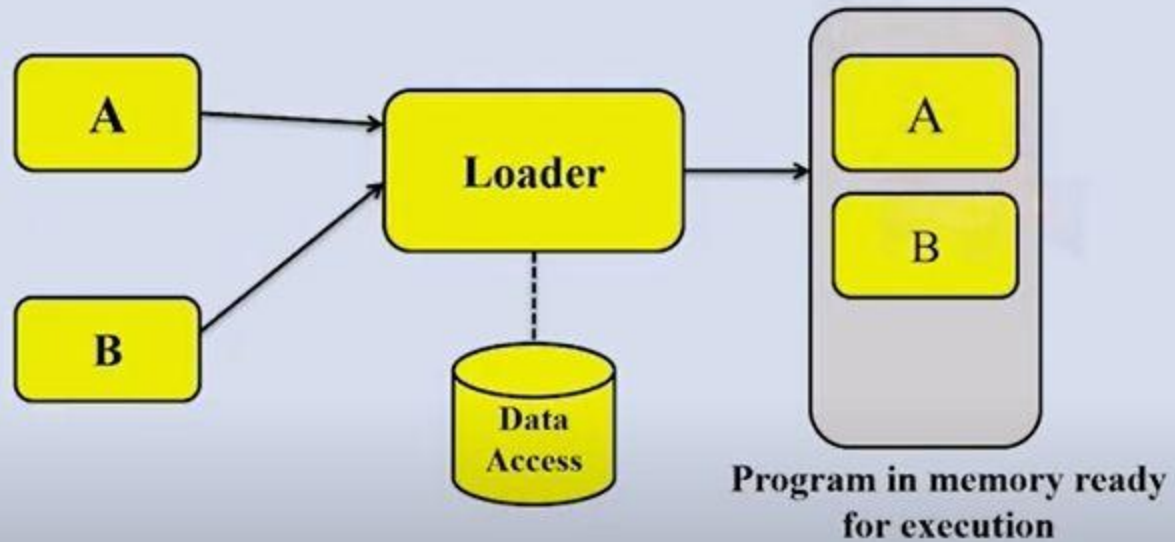


Process of linking a program

Linker

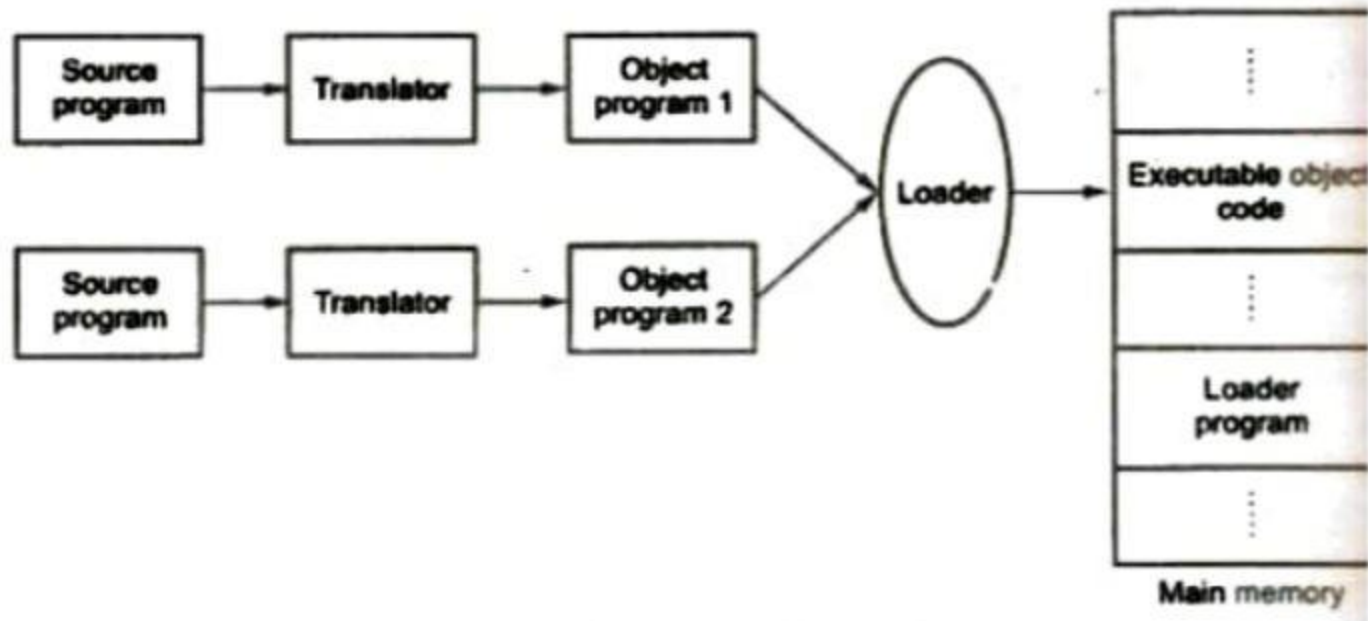


General Loader Scheme



DEFINE LOADER:-

Loader is utility program which takes object code as input prepares it for execution and loads the executable code into the memory. Thus loader is actually responsible for initiating the execution process.



General loader scheme

Absolute Loader

- Absolute loader is a one kind of loader in which relocated object files are created
- Loader accepts these files and places them at a specified location in the memory
- Called Absolute loader because no relocating information is needed, rather obtained from the programmer or assembler.

Absolute Loader

- In this scheme, the translator(e.g. assembler or compiler)outputs the machine language
- Translation of the source program in almost same form as in the compile and go loader
- Absolute loader accepts this translated output in the form of records

Absolute Loader

- **Records containing**
 - ✓ The text (instructions, data or both) to be loaded
 - ✓ The length of text
 - ✓ The starting address

Absolute Loader

- The task of absolute loader is to place these record into the primary memory at the location specified by assembler during process of assembly
- In this scheme, the programmer or assembler should have knowledge of memory management.
- The resolution of external references or linking of different subroutines are the issues which need to be handled by the programmer.

Absolute Loader

- The programmer should take care of two things: first thing is specification of starting address of each module to be used..
- Second thing is, while branching from one segment to another the absolute starting address of respective module is to be known by the programmer.

Absolute Loader

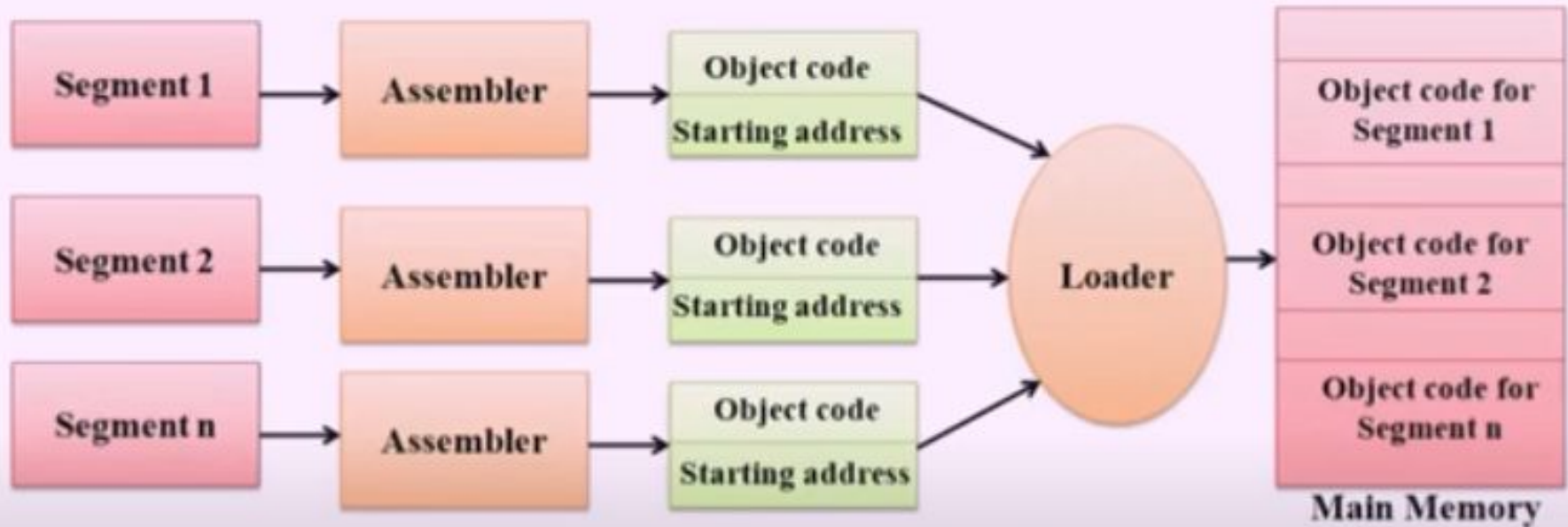


Fig. Process of Absolute Loader[2]

Absolute Loader

- Absolute loader is simple to implement in this scheme
 - ✓ Allocation is done by programmer
 - ✓ Linking is done by the programmer
 - ✓ Relocation is done by assembler
 - ✓ Loading is done by the loader

Absolute Loader

- **Advantages**

- ✓ Simple to implement
- ✓ Task of loader becomes simpler
- ✓ Allows multiple programs or the source programs written in different languages
- ✓ Process of Execution is efficient

Absolute Loader

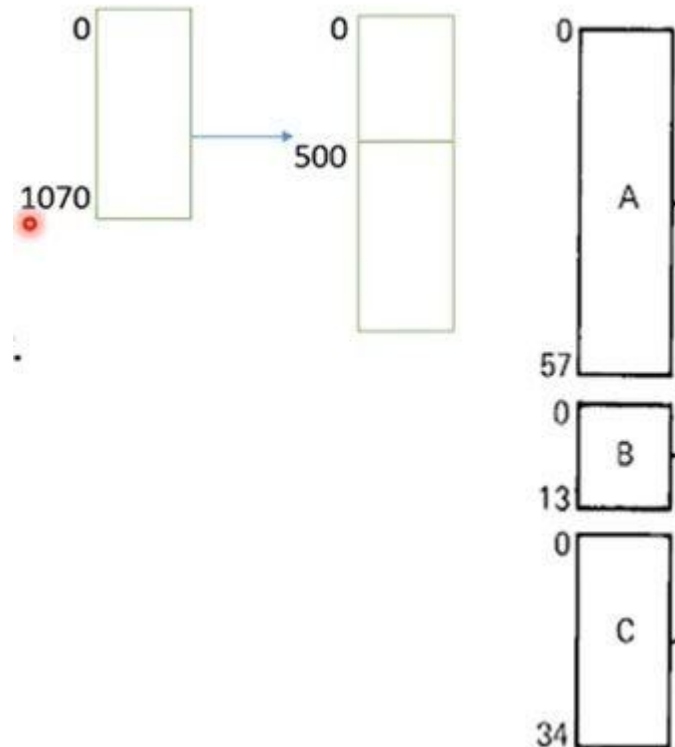
- **Disadvantages**

- ✓ Programmer must specify the loading address in the assembly program
- ✓ In the case of multiple subroutines the programmer must remember the address of each subroutine

Absolute Loader

Absolute loader has several potential disadvantages.

- The **actual address** at which it will be loaded into memory.
- **Cannot run several independent programs** together, sharing memory between them.
- It difficult to use subroutine libraries efficiently.



Relocating Loader

- To avoid possible reassembling of all subroutines when a single subroutine is changed and to perform the task of allocation and linking for the programmer the relocating loaders are used
- The execution of the object program is done using any part of the available and sufficient memory
- The object program is loaded into memory wherever there is room for it

Relocating Loader

- The assembler assembles each procedure segment independently and passes to loader the text and information as to relocation and intersegment references
- The assembler would also provide the loader with additional information, such as the length of entire program and the length of transfer vector

Relocating Loader

- **Transfer Vector**
 - ✓ It is used for linking.
 - ✓ It is the beginning part of the program
 - ✓ It stores names of external subroutines
- Initially when we call the subroutine, it transfers to the corresponding location of transfer vector
- Finally loader calculates absolute location(exact position)



Relocating Loader

- Relocation bit
- It is used for relocation
- This bit is present in object code
- This bit is associated with each instruction
- If this bit is 1 ,then relocation is done
- If this bit is 0,then relocation is not done

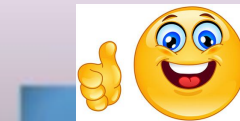
Practical Relocating Loader

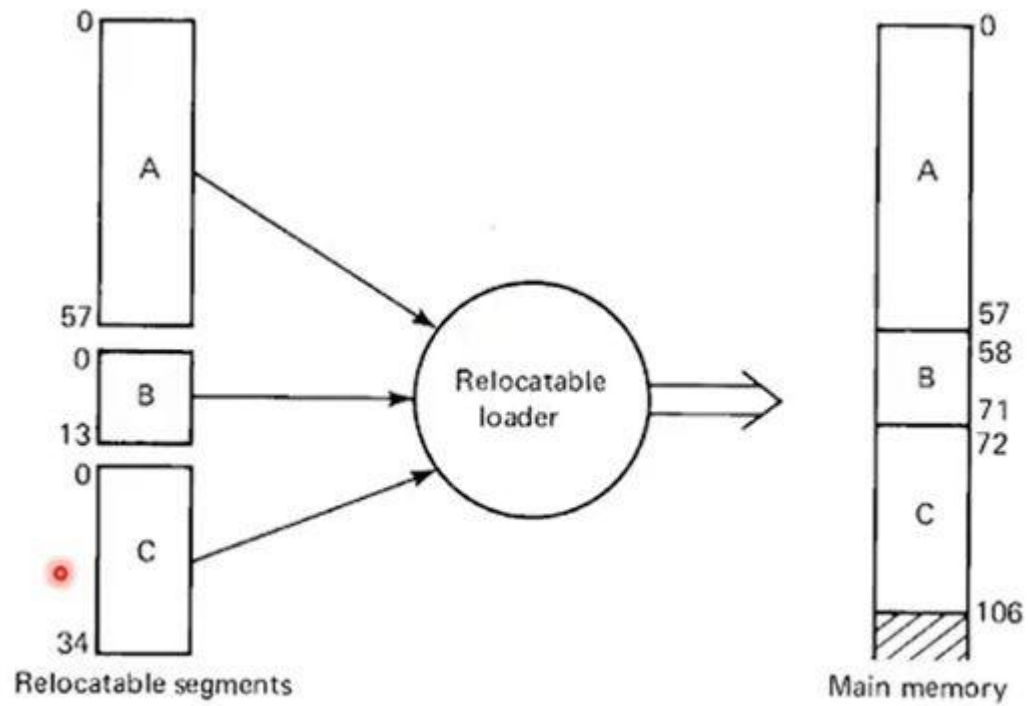
- **Advantages**

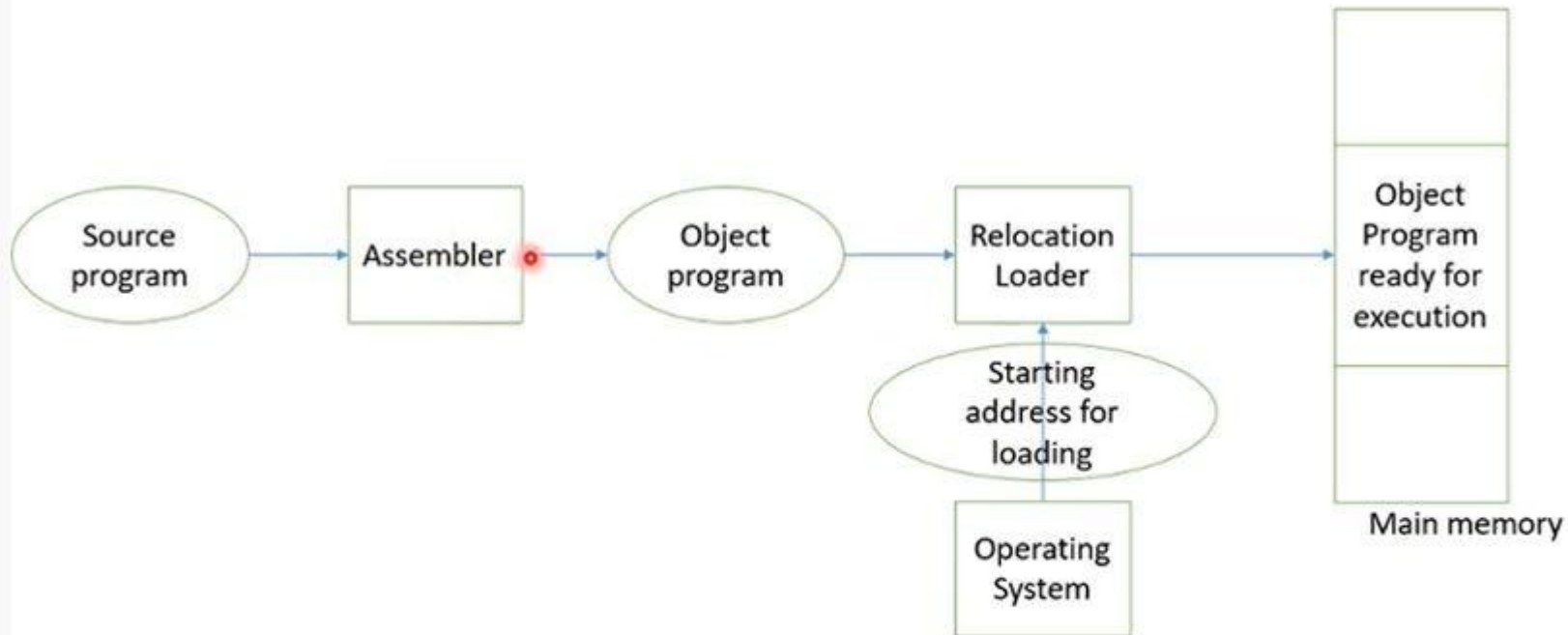
- ✓ Avoids possible reassembling of all subroutines when a single Subroutine is changed
- ✓ Perform the tasks of allocation and linking for the programmer

- **Disadvantages**

- ✓ Difficult to implement
- ✓ Algorithm is depends on File structure of the object program
- ✓ Slower than Absolute loader



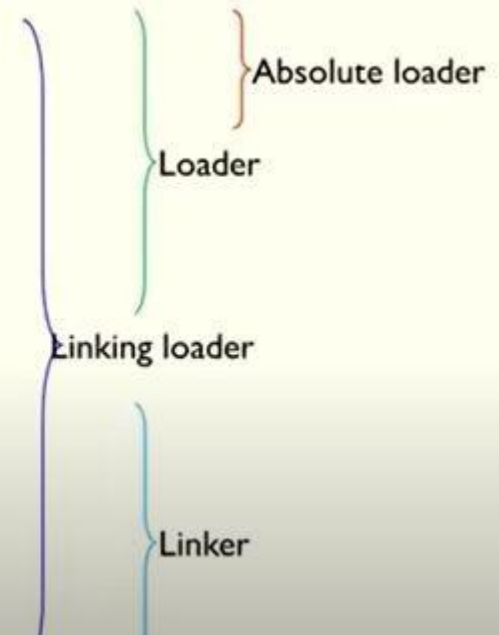




1. Read the object program
2. Get the starting address in memory for storage from operating system
3. Store the object codes from object program to main memory in continuously.
(updated / actual)

Process to run an object program

- Loading
 - Brings object program into memory
- Relocation
 - Modifies the object program where absolute addresses are specified
- Linking
 - Combines two or more separate object programs and supplies information needed to allow cross-references.

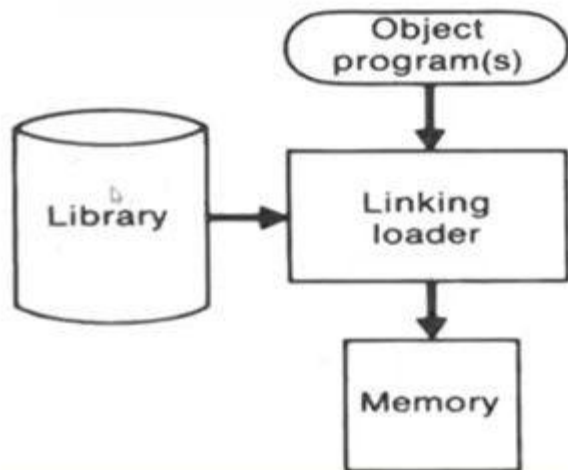


- In particular Loader must perform following functions:
- Allocate space in memory for the programs (allocation).
 - Resolve symbolic references between object decks (linking).
 - Adjust all address dependent locations, such as address constants , to correspond to the allocated space(relocation).

Design of Linker

- 1. Linking Loader
- 2. Linkage Editor
- Dynamic Linking

Linking Loader



- Each time program calls the library function it will loaded into memory for execution by linking loader

Disadvantages of Direct Linking

- It is necessary to allocate, relocate, link, and load all of the subroutines each time in order to execute a program
 - loading process can be extremely time consuming.
- Though smaller than the assembler, the loader absorbs a considerable amount of space
 - Dividing the loading process into two separate programs a binder and a module loader can solve these problems.

Linkage Editor

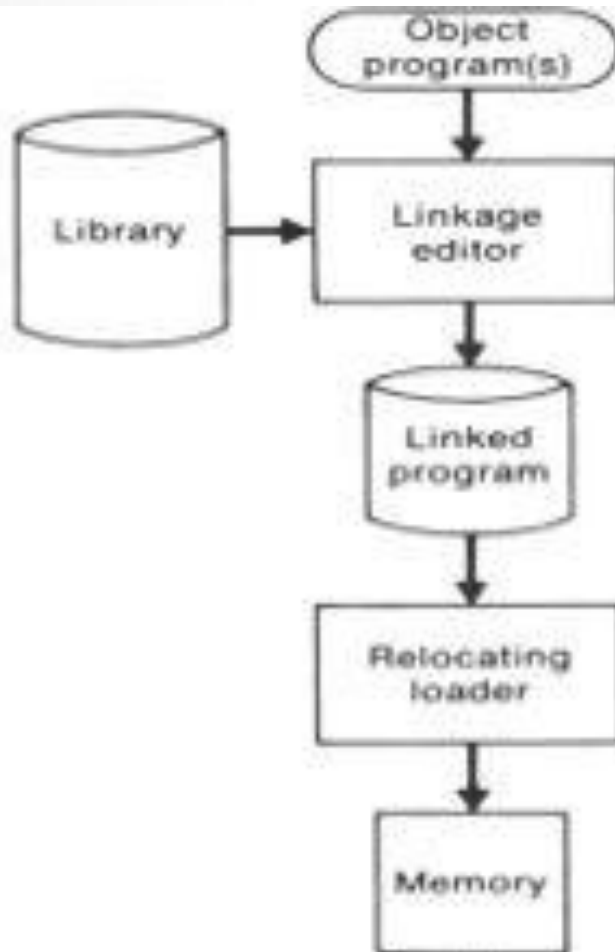
Linking loader

- performs all **linking** and **relocation** operations, including automatic library search, and **loads** the linked program into memory for execution.

Linkage editor

- **produces a linked version** of the program, which is normally written to a file or library for later execution.
 - A simple **relocating loader** (one pass) can be used to load the program into memory for execution.
 - The linkage editor performs relocation of all control sections relative to the start of the linked program.
 - The only object code modification necessary is the addition of an actual load address to relative values within the program.

Linkage Editor



- If the object program calls the library function, the linkage editor will call the library and create the linked version of the program. So next when the same is required the loader will call this linked program so no need to load the library function again and again. If program is modified then I have to re-run it.

Difference between a linkage editor and a linking loader:

– Linking loader

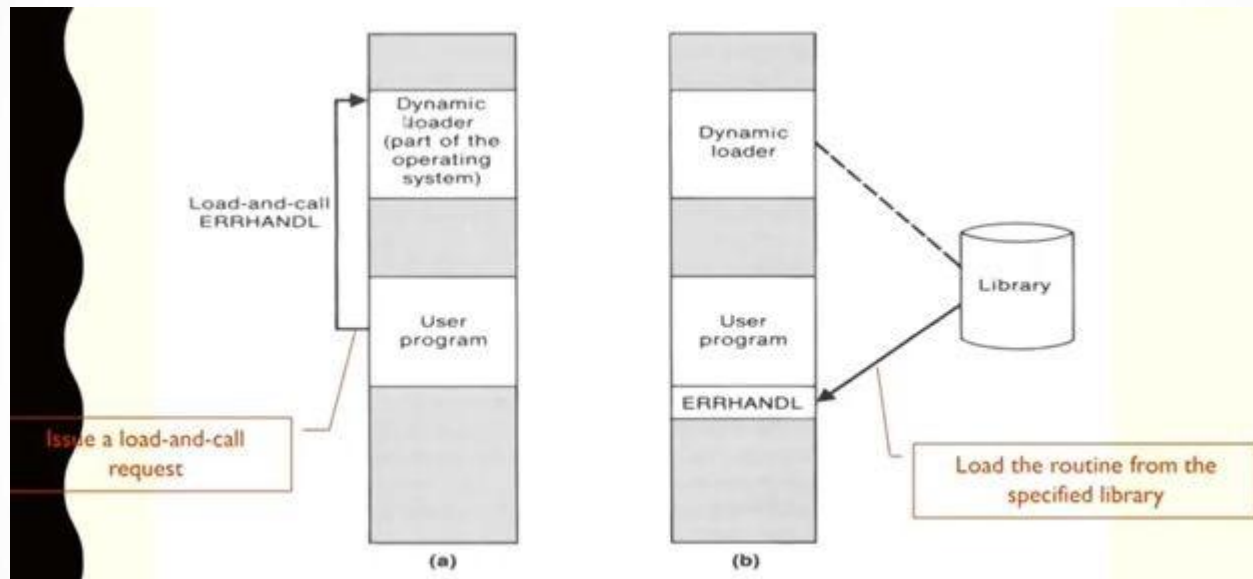
- Suitable when a program is reassembled for nearly every execution
 - In a program development and testing environment
 - When a program is used so infrequently that it is not worthwhile to store the assembled and linked version.

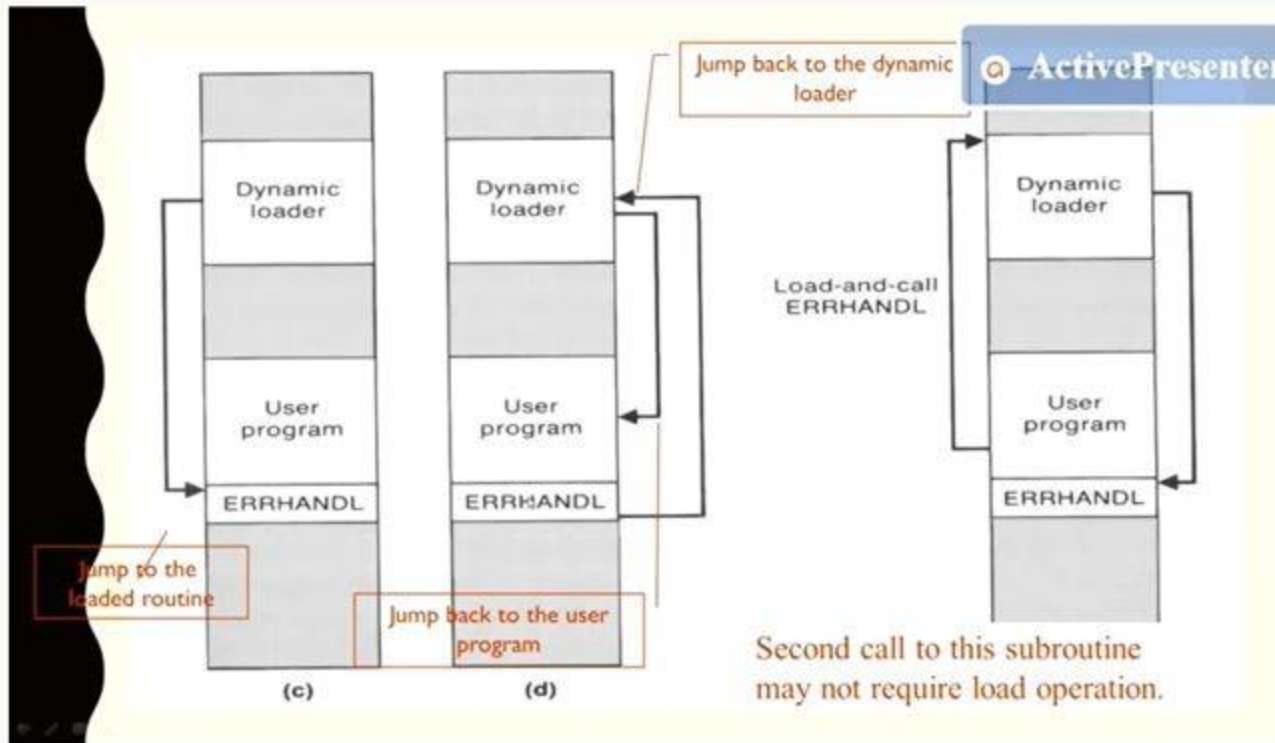
– Linkage editor

- Suitable when a program is to be executed many times without being reassembled because resolution of external references and library searching are only performed once.

Dynamic Linking

- Subroutine to be dynamically loaded by OS service request
e.g. load and call
- The service request will be handled by a part of OS called the dynamic loader which is kept in memory during execution of the program
- The parameter of this request is the symbolic name of the routine to be called



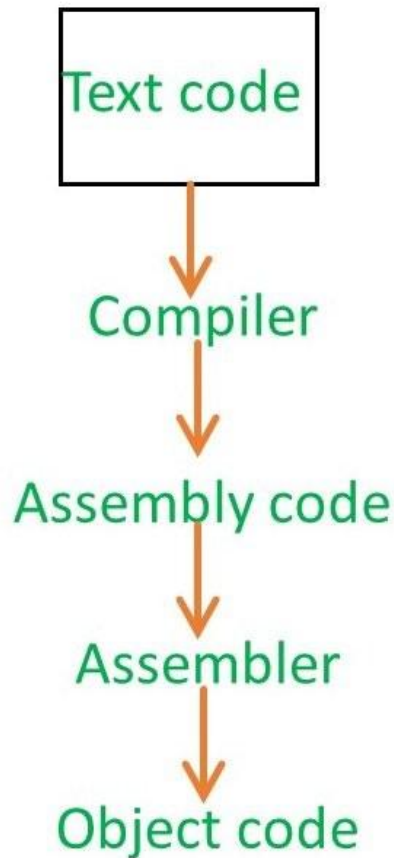


COMPARISON

- Linkage editors: **before** load time
- Linking loaders: **at** load time
- Dynamic linking: **after** load time
 - A scheme that postpones the linking function until execution time.
 - A subroutine is loaded and linked to the program when it is first called.
 - Other names: *dynamic loading, load on call*



Object Code



- Give the C program to compiler and compiler will produce the output in assembly code. Now, that assembly language code will give to the assembler and assembler is going to produce you some code. That is known as **Object**

Object Code

Header
Text segment
Data segment
Relocation information
Symbol table
Debugging information

OBJECT CODE

- The compiler is actually combined inside the assembler along with loader and linker. All the module kept together in the compiler software itself. So when you calling gcc, you are calling the compiler, then assembler, then linker and loader.
- Once you call the compiler, then your object code is going to present in Hard-disk. This object code contains various part – as shown in fig.

Header

- The header is like an index and will tell you, what are the places at which each information is present. Header will say where the text segment is going to start and a pointer to it and where the data segment going to start and it say where the relocation information and symbol information there.

- **Text segment** –
It is the set of
instruction.

- **Data segment** –
Data segment will
contain whatever data
you have used.

Relocation Information

0 L0 level

1 L1 level

2 L2 level

3 L3 level

4 L4 level

5 L5 level

Goto L4

Goto 4

- Let us assume you have instruction 1, instruction 2, instruction 3, instruction 4,...
- if you say Goto L4
- Goto 4 for the level L4 is going to work fine, as long as the program is going to be loaded starting at address no 0. But in most cases, the initial part of the RAM is going to be dedicated to the operating system or might be some other process that will already be running at address no 0. So, program might be loaded anywhere. Let us say 1000 is the new starting address, then all the addresses have to be changed, that is known as **Reallocation**.

1000 L0 level
1001 L1 level
1002 L2 level
1003 L3 level
1004 L4 level
1005 L5 level

Goto

1004

Absolute
address

0 L0 level
1 L1 level
2 L2 level
3 L3 level
4 L4 level
5 L5 level

Goto L4

Goto

4

Relocatable
address

Relocation



Relocation Information

- The original address is known as **Relocatable address** and the final address which we get after loading the program into main memory is known as the **Absolute address**.

- **Symbol table –**

It contains every symbol that you have in your program. for example, `int a, b, c` then, `a, b, c` are the symbol. It will show what are the variables that your program contains.

- **Debugging information –**

It will help to find how a variable is keeping on changing.

- We can classify program depending on the manner in which a program can modify or not modify itself to execute from a given load origin into following categories
- Relocatable program- done by linkage editor
relocatable loader, size of program > Non relocatable program
- Self relocatable program
- Non relocatable program

There are

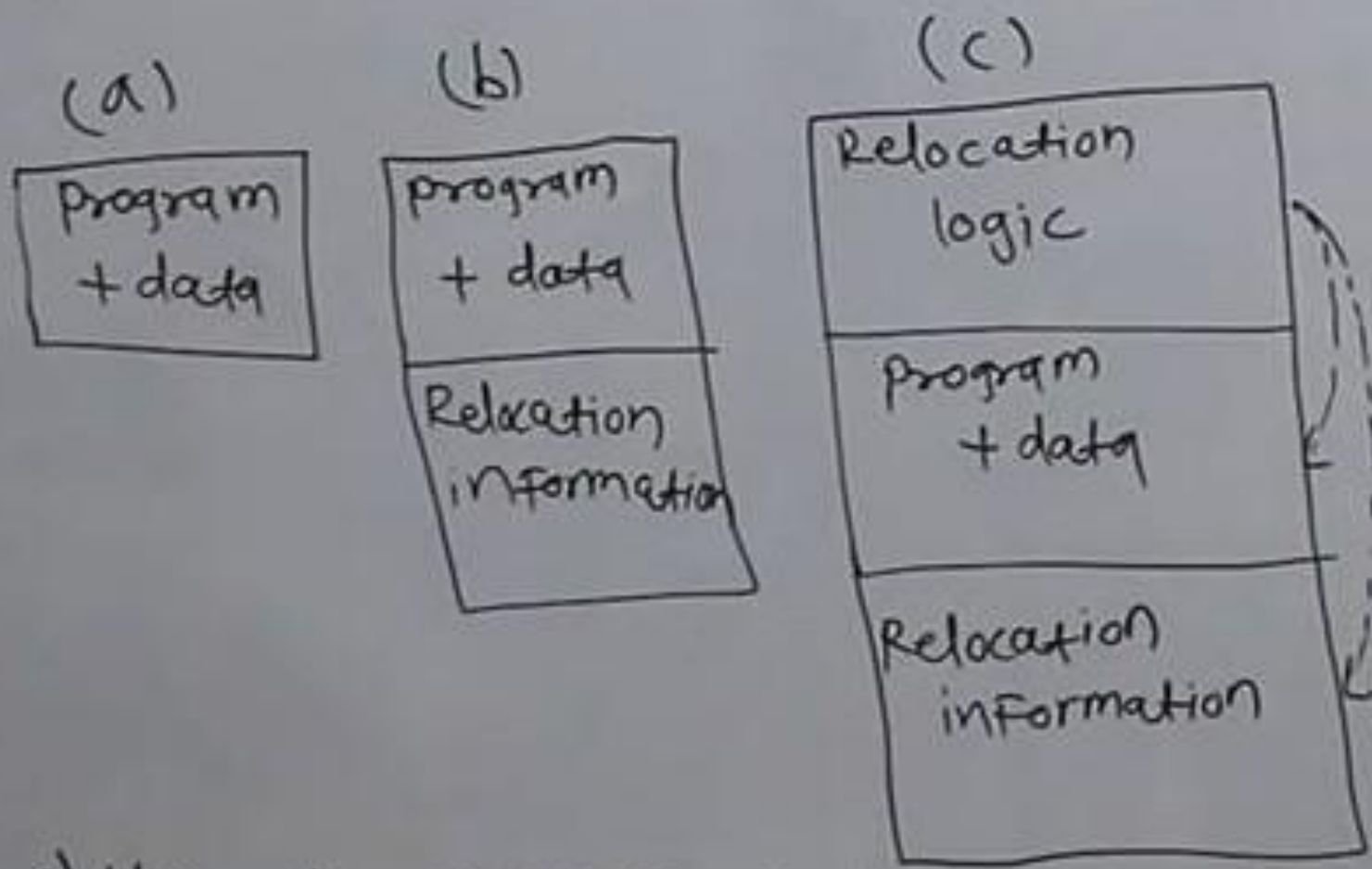


Fig. a) Non-rel. prog^m, b) Rel. prog^m, c) Self-rel. prog^m.

RELOCATABLE PROGRAM

Relocatable program is one that can be read into memory at any address and executed without modification. Or A Relocatable program form is one which consists of a program and relevant information for its relocation. Using this information it is possible to relocate the program to execute from a storage area then the one designated for it at the time of its coding or translation. Relocatable program

Non relocatable program

- Program can not be made to execute anywhere in storage other than area starting on its translated origin. The code is address sensitive and lack info like which part of the program is address sensitive and in which manner.
- Here load time origin= translation origin also link address are same

Self relocatable program

- A program which perform relocation of its own address sensitive instruction.
- Code has a table of information related to address sensitive instruction called relocation info.
- A start of relocating logic address is always specified as execution start address of any program
- By use of relocating logic and info of all address sensitive instruction it perform relocation by itself

Types of Loading

- ❖ Static Loading
- ❖ Dynamic Loading

What is Static Loading ?

- ❖ Loading the entire program into the main memory before start of the program execution is called as static loading.

- ❖ **Inefficient utilization of memory in Static Loading. Why ?**

Because whether it is required or not required entire program is brought into the main memory.

- ❖ Program execution will be faster.

- ❖ Statically linked program takes constant load time every time it is loaded into the memory for execution.

- ❖ If the static loading is used then accordingly static linking is applied.

What is Dynamic Loading ?

- ❖ Loading the program into the main memory on demand is called as dynamic loading.
- ❖ Efficient utilization of memory.
- ❖ Program execution will be slower.

•



Types of Linking

- ❖ Static Linking
- ❖ Dynamic Linking

What is Static Linking ?

- ❖ Static linking is performed by programs called linkers as the last step in compiling a program.
- ❖ Linkers are also called link editors.
- ❖ If any of the external programs has changed then they have to be recompiled and re-linked again else the changes won't reflect in existing executable file.

What is Dynamic Linking ?

- ❖ Dynamic linking is performed at run time by the operating system.
- ❖ If the dynamic loading is used then accordingly dynamic linking is applied.
- ❖ In dynamic linking this is not the case and individual shared modules can be updated and recompiled. This is one of the greatest advantages dynamic linking offers.
- ❖ In dynamic linking load time might be reduced if the shared library code is already present in memory.

What is Relocation ?

- ❖ Relocation is the process of assigning load addresses for position-dependent code and data of a program and adjusting the code and data to reflect the assigned addresses.
- ❖ Prior to the advent of multiprocess systems, and still in many embedded systems, the addresses for objects were absolute starting at a known location, often zero.
- ❖ Since multiprocessing systems dynamically link and switch between programs it became necessary to be able to relocate objects using position-independent code.

Thank You !!!